Sorting of data

12 December 2024 20:14

What is Sorting?
To arrange the values either in ascending order or in descending order.

How it helps?
Sorting helps us to find the required element as quickly as possible or in a convenient manner.

There are many algorithms to sort the data:

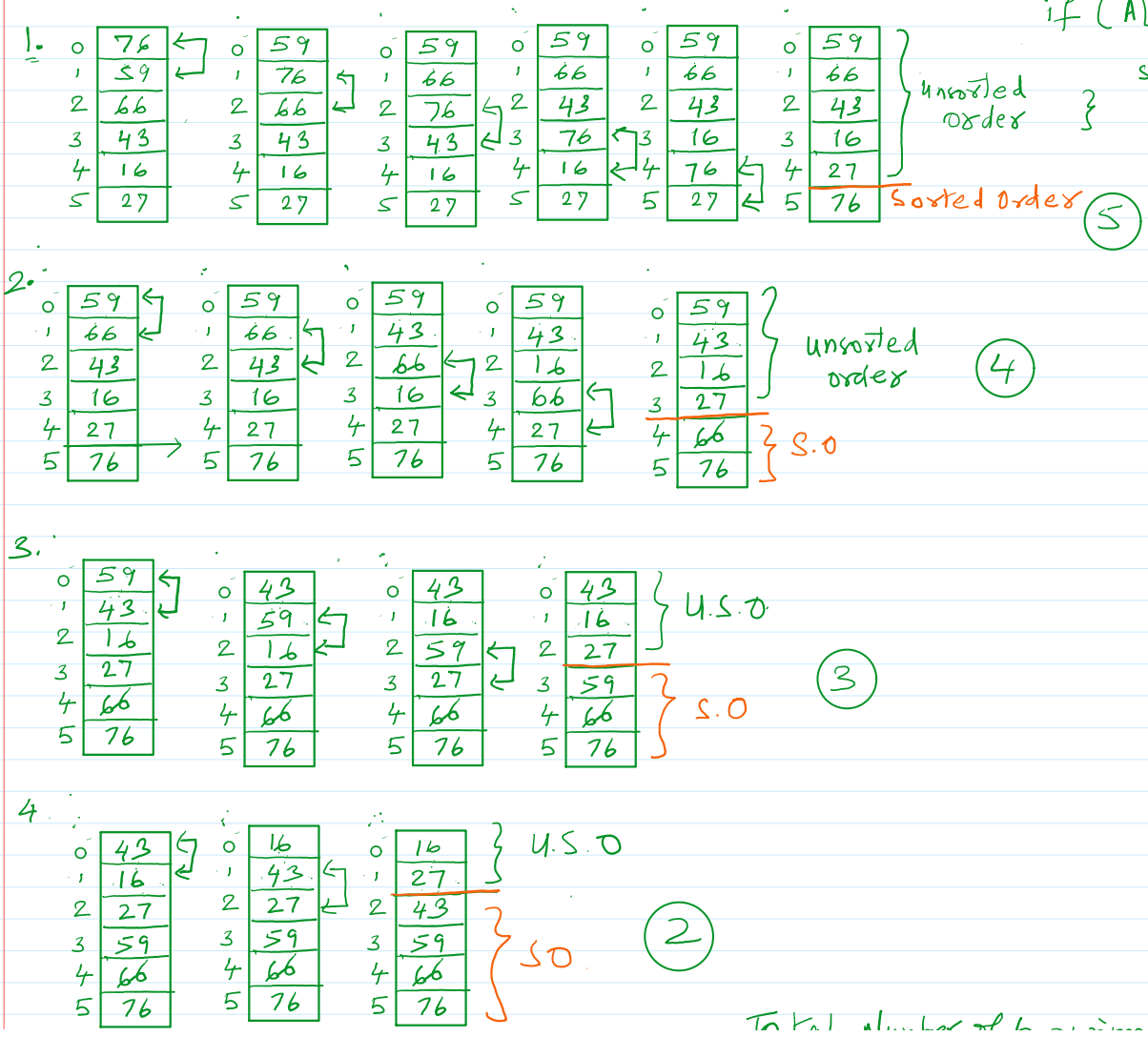
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Merge Sort
5. Quick Sort
6. Shell Sort
7. Heap Sort
8. Tim Sort
9. Index Sort or Radix Sort

} fundamental sorting technique.

Bubble Sort

In Bubble Sort we compare adjacent elements and swap the elements depending upon a condition. The sorted element sediments one upon the other, and the unsorted elements bubbles up. In one pass only one element is in sorted order. If we have N elements then we need N-1 passes to sort all the elements. For example:

if (A[j] > A[j+1]) {
swap (A[j], A[j+1])
}



Total number of passes = 5+4+3+2+1 = 15

3	59	3	21	3	59
4	66	4	66	4	66
5	76	5	76	5	76

SO

Total Number of Comparism $= 5+4+3+2+1 = 15$

5.

0	16	0	16		
1	27	1	27		
2	43	2	43		
3	59	3	59		
4	66	4	66		
5	76	5	76		

S.O.

1

To sort 6 elements we need 15 steps.
7 elements we need $(15+6) = 21$ steps.
8 elements we need $(21+7) = 28$ steps
9 elements we need $28+8 = 36$ steps
10 elements we need $36+9 = 45$ steps
⋮
N elements we need.

$= (N-1) + (N-2) + (N-3) + \dots + 1$

To sort 'N' elements $= \frac{N(N-1)}{2}$ steps needed.

$= \frac{\cancel{8}\times 7}{\cancel{2}} = 28$ steps

$1+2+3+4+5+6 = 21$

Sum of Natural numbers: $\frac{N\times(N+1)}{2}$

$= \frac{\cancel{6}*7}{\cancel{2}} = 21$ $\quad [\because N = (n-1)]$

$\therefore \frac{(n-1)*(\cancel{n-1}-\cancel{1})}{2} = \frac{n(n-1)}{2}$.

$= \frac{n(n-1)}{2}$ steps needs

$= \frac{n^2-n}{2} = \boxed{\frac{1}{2}n^2 - \frac{1}{2}n^1}$.

constant or coefficient

$\frac{1}{2}$ n^2 variable.

Big-O function: In this function we select only the major term or the term with highest degree, without any constant.

Performance of Bubble sort algorithm $\approx O(n^2)$

data (n)	Steps $= O(n^2)$
10	$10^2 = 100$
100	$100^2 = 10000$
1000	$1000^2 = 1000000$

As number of element increses, the time also increases exponentially. Hence, bubble sort is good for less number of elements.

```
public void bubbleSort (int A[]) {
    int i, j, t;
```

```
public void bubbleSort(int A[]){
    int i, j, t;
    for(i=0; i < A.length-1; i++){
        for(j=0; j< A.length-1-i; j++){
            if(A[j] > A[j+1]){
                t = A[j]; A[j]=A[j+1]; A[j+1]=t;
            }
        }
    }
}
```

```
package ArraysInJava;
public class SortingAlgorithms
{
    public static void bubbleSort(int A[]){
        int i, j, t;
        for(i = 0; i < A.length - 1; i++){
            for(j = 0; j < A.length - 1 - i; j++){
                if(A[j] > A[j+1]){
                    t = A[j];
                    A[j] = A[j+1];
                    A[j+1] = t;
                }
            }
        }
    }
}
package ArraysInJava;
import java.util.*;


/**
 * Write a description of class ArrayUtility here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ArrayUtility
{
    public static void inputRandomValues(int A[]){
        int i;
        Random R = new Random();
        for(i = 0; i < A.length; i++){
            A[i] = R.nextInt(10, 100);
        }
    }

    public static void printArray(int A[], String msg){
        System.out.print("\n" + msg + "\n[");
        int i;
        for(i = 0; i < A.length - 1; i++){
            System.out.print(A[i] + ", ");
        }
        System.out.print(A[i]+"]\n");
    }
}

package ArraysInJava;
import java.util.*;
```

```
/**
 * Write a description of class TestSorting here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestSorting
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter number of elements: ");
        int N = sc.nextInt();
        int A[] = new int[N];
        ArrayUtility.inputRandomValues(A);
        ArrayUtility.printArray(A, "Array A[] before sorting: ");
        SortingAlgorithms.bubbleSort(A);
        ArrayUtility.printArray(A, "Array A[] after sorting: ");
    }
}
```

Home work: Write descending order logic to sort the array in descending order.